

LAB PROGRAMME
CHAPTER 5 : GREEDY

NAME: S. BHUVANESH

REG NO: 192521004

1. There are $3n$ piles of coins of varying size, you and your friends will take piles of coins as follows: In each step, you will choose any 3 piles of coins (not necessarily consecutive). Of your choice, Alice will pick the pile with the maximum number of coins. You will pick the next pile with the maximum number of coins. Your friend Bob will pick the last pile. Repeat until there are no more piles of coins. Given an array of integers piles where piles[i] is the number of coins in the i-th pile. Return the maximum number of coins that you can have.

Example 1:

Input: piles = [2,4,1,2,7,8]

Output: 9

Explanation: Choose the triplet (2, 7, 8), Alice Pick the pile with 8 coins, you the pile with 7 coins and Bob the last one.

Choose the triplet (1, 2, 4), Alice Pick the pile with 4 coins, you the pile with 2 coins and Bob the last one.

The maximum number of coins which you can have is: $7 + 2 = 9$.

On the other hand if we choose this arrangement (1, 2, 8), (2, 4, 7) you only get $2 + 4 = 6$ coins which is not optimal.

Example 2:

Input: piles = [2,4,5]

Output: 4

```
main.py
1 def maxCoins(piles: list[int]) -> int:
2     # 1. Sort the piles in ascending order
3     piles.sort()
4
5     # 2. Determine the number of rounds (n)
6     n = len(piles) // 3
7     total_coins = 0
8
9     # 3. Start from the second-largest element (index len-2)
10    # and pick every second element moving backwards for n rounds
11    for i in range(len(piles) - 2, n - 1, -2):
12        total_coins += piles[i]
13
14    return total_coins
15
16 # --- Test Cases ---
17 print(maxCoins([2, 4, 1, 2, 7, 8])) # Output: 9
18 print(maxCoins([2, 4, 5]))          # Output: 4
19 print(maxCoins([9, 8, 7, 6, 5, 1, 2, 3, 4])) # Output: 18
20
21
22
```

input

9
4
18

2. You are given a 0-indexed integer array `coins`, representing the values of the coins available, and an integer `target`. An integer `x` is obtainable if there exists a subsequence of coins that sums to `x`. Return the minimum number of coins of any value that need to be added to the array so that every integer in the range `[1, target]` is obtainable. A subsequence of an array is a new non-empty array that is formed from the original array by deleting some (possibly none) of the elements without disturbing the relative positions of the remaining elements.

Example 1:

Input: `coins = [1,4,10]`, `target = 19`

Output: 2

Explanation: We need to add coins 2 and 8. The resulting array will be `[1, 2, 4, 8, 10]`.

It can be shown that all integers from 1 to 19 are obtainable from the

resulting array, and that 2 is the minimum number of coins that need to be added to the array.

Example 2:

Input: coins = [1, 4, 10, 5, 7, 19], target = 19

Output: 1

Explanation: We only need to add the coin 2. The resulting array will be [1,2, 4, 5, 7, 10, 19].

It can be shown that all integers from 1 to 19 are obtainable from the resulting array, and that 1 is the minimum number of coins that need to be added to the array

```
main.py
1 def minCoinsAdded(coins: list[int], target: int) -> int:
2     coins.sort()
3     miss = 1
4     added = 0
5     i = 0
6
7     while miss <= target:
8         if i < len(coins) and coins[i] <= miss:
9             miss += coins[i]
10            i += 1
11        else:
12            miss += miss
13            added += 1
14
15    return added
16
17 # Test Verification
18 print(minCoinsAdded([1, 4, 10], 19)) # Output: 2
19 print(minCoinsAdded([1, 4, 10, 5, 7, 19], 19)) # Output: 1
20
21
22
```

input

2

1

3. You are given an integer array jobs, where jobs[i] is the amount of time it takes to complete the ith job. There are k workers that you can assign jobs to. Each job should be assigned to exactly one worker. The working time of a worker is the sum of the time it takes to complete all jobs assigned to them. Your goal is to devise an optimal assignment such that the maximum

working time of any worker is minimized. Return the minimum possible maximum working time of any assignment.

Example 1:

Input: jobs = [3,2,3], k = 3

Output: 3

Explanation: By assigning each person one job, the maximum time is 3.

Example 2:

Input: jobs = [1,2,4,7,8], k = 2

Output: 11

Explanation: Assign the jobs the following way:

Worker 1: 1, 2, 8 (working time = $1 + 2 + 8 = 11$)

Worker 2: 4, 7 (working time = $4 + 7 = 11$)

The maximum working time is 11.

```
main.py
1 def minimumTimeRequired(jobs: list[int], k: int) -> int:
2     jobs.sort(reverse=True)
3     workloads = [0] * k
4     self_res = float('inf')
5
6     def backtrack(idx):
7         nonlocal self_res
8         if idx == len(jobs):
9             self_res = min(self_res, max(workloads))
10            return
11
12            if max(workloads) >= self_res:
13                return
14
15            seen = set()
16            for i in range(k):
17                if workloads[i] in seen:
18                    continue
19                if workloads[i] + jobs[idx] >= self_res:
20                    continue
21                seen.add(workloads[i])
22                workloads[i] += jobs[idx]
23                backtrack(idx + 1)
24                workloads[i] -= jobs[idx]
25
26            return self_res
27
28 return backtrack(0)
```

3

11

input

4. We have n jobs, where every job is scheduled to be done from $\text{startTime}[i]$ to $\text{endTime}[i]$, obtaining a profit of $\text{profit}[i]$. You're given the startTime , endTime and profit arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range. If you choose a job that ends at time X you will be able to start another job that starts at time X .

Example 1:

Input: $\text{startTime} = [1,2,3,3]$, $\text{endTime} = [3,4,5,6]$, $\text{profit} = [50,10,40,70]$

Output: 120

Explanation: The subset chosen is the first and fourth job.

Time range $[1-3]+[3-6]$, we get profit of $120 = 50 + 70$.

Example 2:

Input: $\text{startTime} = [1,2,3,4,6]$, $\text{endTime} = [3,5,10,6,9]$, $\text{profit} = [20,20,100,70,60]$

Output: 150

Explanation: The subset chosen is the first, fourth and fifth job. Profit obtained $150 = 20 + 70 + 60$.

```
main.py
1 import bisect
2
3 def jobScheduling(startTime: list[int], endTime: list[int], profit: list[int]) -> int:
4     jobs = sorted(zip(startTime, endTime, profit), key=lambda x: x[1])
5     dp = [(0, 0)] # (endTime, profit)
6
7     for s, e, p in jobs:
8         # Find the latest non-overlapping job using binary search
9         idx = bisect.bisect_right(dp, (s, float('inf'))) - 1
10        current_profit = dp[idx][1] + p
11        if current_profit > dp[-1][1]:
12            dp.append((e, current_profit))
13
14    return dp[-1][1]
15
16 # Test Verification
17 print(jobScheduling([1, 2, 3, 3], [3, 4, 5, 6], [50, 10, 40, 70]))
18 print(jobScheduling([1, 2, 3, 4, 6], [3, 5, 10, 6, 9], [20, 20, 100, 70, 60]))
19
```

input

120
150

5. Given a graph represented by an adjacency matrix, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to all other vertices in the graph. The graph is represented as an adjacency matrix where $graph[i][j]$ denote the weight of the edge from vertex i to vertex j . If there is no edge between vertices i and j , the value is Infinity (or a very large number).

Test Case 1:

Input:

$n = 5$

$graph = [[0, 10, 3, \text{Infinity}, \text{Infinity}], [\text{Infinity}, 0, 1, 2, \text{Infinity}], [\text{Infinity}, 4, 0, 8, 2],$

$[\text{Infinity}, \text{Infinity}, \text{Infinity}, 0, 7], [\text{Infinity}, \text{Infinity}, \text{Infinity}, 9, 0]]$

$source = 0$

Output: $[0, 7, 3, 9, 5]$

Test Case 2:

Input:

$n = 4$

$graph = [[0, 5, \text{Infinity}, 10], [\text{Infinity}, 0, 3, \text{Infinity}], [\text{Infinity}, \text{Infinity}, 0, 1],$

$[\text{Infinity}, \text{Infinity}, \text{Infinity}, 0]]$

$source = 0$

Output: $[0, 5, 8, 9]$

```
main.py
1 import heapq
2
3 def dijkstra_matrix(n: int, graph: list[list[float]], source: int) -> list[float]:
4     distances = [float('inf')] * n
5     distances[source] = 0
6     min_heap = [(0, source)]
7
8     while min_heap:
9         dist, u = heapq.heappop(min_heap)
10
11         if dist > distances[u]:
12             continue
13
14         for v in range(n):
15             weight = graph[u][v]
16             if weight != float('inf') and u != v:
17                 if distances[u] + weight < distances[v]:
18                     distances[v] = distances[u] + weight
19                     heapq.heappush(min_heap, (distances[v], v))
20
21     return distances
22
23 # Test Verification
24 inf = float('inf')
25 g1 = [[0, 10, 3, inf, inf], [inf, 0, 1, 2, inf], [inf, 4, 0, 8, 2], [inf, inf, inf, 0, 7], [inf, inf, inf, 9, 0]]
26 print(dijkstra_matrix(5, g1, 0)) # Output: [0, 7, 3, 9, 5]
27
```

input

[0, 7, 3, 9, 5]

6. Given a graph represented by an edge list, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to a target vertex. The graph is represented as a list of edges where each edge is a tuple (u, v, w) representing an edge from vertex u to vertex v with weight w.

Test Case 1:

Input:

n = 6

edges = [(0, 1, 7), (0, 2, 9), (0, 5, 14), (1, 2, 10), (1, 3, 15),
(2, 3, 11), (2, 5, 2), (3, 4, 6), (4, 5, 9)]

source = 0

target = 4

Output: 20

Test Case 2:

Input:

n = 5

edges = [(0, 1, 10), (0, 4, 3), (1, 2, 2), (1, 4, 4), (2, 3, 9), (3, 2, 7), (4, 1, 1),
(4, 2, 8), (4, 3, 2)]

source = 0

target = 3

Output: 8

```

main.py
1 import heapq
2 from collections import defaultdict
3
4 def dijkstra_edge_list(n: int, edges: list[tuple], source: int, target: int) -> float:
5     graph = defaultdict(list)
6     for u, v, w in edges:
7         graph[u].append((v, w))
8
9     distances = {i: float('inf') for i in range(n)}
10    distances[source] = 0
11    min_heap = [(0, source)]
12
13    while min_heap:
14        dist, u = heapq.heappop(min_heap)
15
16        if u == target:
17            return dist
18
19        if dist > distances[u]:
20            continue
21
22        for v, weight in graph[u]:
23            if distances[u] + weight < distances[v]:
24                distances[v] = distances[u] + weight
25                heapq.heappush(min_heap, (distances[v], v))
26
27    return distances[target]

```

7. Given a set of characters and their corresponding frequencies, construct the Huffman Tree and generate the Huffman Codes for each character.

Test Case 1:

Input:

n = 4

characters = ['a', 'b', 'c', 'd']

frequencies = [5, 9, 12, 13]

Output: [('a', '110'), ('b', '10'), ('c', '0'), ('d', '111')]

Test Case 2:

Input:

n = 6

characters = ['f', 'e', 'd', 'c', 'b', 'a']

frequencies = [5, 9, 12, 13, 16, 45]

Output: [('a', '0'), ('b', '101'), ('c', '100'), ('d', '111'), ('e', '1101'), ('f', '1100')]


```
main.py
16
17 while len(heap) > 1:
18     left = heapq.heappop(heap)
19     right = heapq.heappop(heap)
20     merged = HuffmanNode(None, left.freq + right.freq)
21     merged.left = left
22     merged.right = right
23     heapq.heappush(heap, merged)
24
25 root = heap[0]
26 codes = {}
27
28 def generate_codes(node, current_code):
29     if not node:
30         return
31     if node.char is not None:
32         codes[node.char] = current_code
33         return
34     generate_codes(node.left, current_code + "0")
35     generate_codes(node.right, current_code + "1")
36
37 generate_codes(root, "")
38 return [(ch, codes[ch]) for ch in characters]
39
40 # Test Verification
41 print(build_huffman_tree(['a', 'b', 'c', 'd'], [5, 9, 12, 13]))
42
```

input

```
[('a', '00'), ('b', '01'), ('c', '10'), ('d', '11')]
```

8. Given a Huffman Tree and a Huffman encoded string, decode the string to get the original message.

Test Case 1:

Input:

n = 4

characters = ['a', 'b', 'c', 'd']

frequencies = [5, 9, 12, 13]

encoded_string = '1101100111110'

Output: 'abacd'

Test Case 2:

Input:

n = 6

```
characters = ['f', 'e', 'd', 'c', 'b', 'a']
```

```
frequencies = [5, 9, 12, 13, 16, 45]
```

```
encoded_string = '110011011100101111001011'
```

Output: "fcbade"

```
main.py
5     self.char = char
6     self.freq = freq
7     self.left = None
8     self.right = None
9
10    # Python heap handles comparison using frequency
11    def __lt__(self, other):
12        return self.freq < other.freq
13
14    def decode_huffman(characters: list[str], frequencies: list[int], encoded_string: str) -> str:
15        # 1. Rebuild the Huffman Tree exactly as it was encoded
16        heap = [HuffmanNode(ch, freq) for ch, freq in zip(characters, frequencies)]
17        heapq.heapify(heap)
18
19        while len(heap) > 1:
20            left = heapq.heappop(heap)
21            right = heapq.heappop(heap)
22
23            # Combine frequencies
24            merged = HuffmanNode(None, left.freq + right.freq)
25            merged.left = left
26            merged.right = right
27            heapq.heappush(heap, merged)
28
29        root = heap[0]
30
31        # 2. Decode the binary string by traversing the tree
32        decoded_output = []
33        current_node = root
34
35        for bit in encoded_string:
```

```
Test Case 1 Output: dbcbdd
Test Case 2 Output: fefcbaac
```

9. Given a list of item weights and the maximum capacity of a container, determine the maximum weight that can be loaded into the container using a greedy approach. The greedy approach should prioritize loading heavier items first until the container reaches its capacity.

Test Case 1:

Input:

n = 5

```
weights = [10, 20, 30, 40, 50]
```

```
max_capacity = 60
```

Output: 50

Test Case 2:

Input:

n = 6

weights = [5, 10, 15, 20, 25, 30]

max_capacity = 50

Output: 50

```
main.py
1 def max_weight_container(weights: list[int], max_capacity: int)
2     sorted_weights = sorted(weights, reverse=True)
3     current_weight = 0
4
5     for w in sorted_weights:
6         if current_weight + w <= max_capacity:
7             current_weight += w
8
9     return current_weight
10
11 # Test Verification
12 print(max_weight_container([10, 20, 30, 40, 50], 60)) # Output
13
14
```

10. Given a list of item weights and a maximum capacity for each container, determine the minimum number of containers required to load all items using a greedy approach. The greedy approach should prioritize loading items into the current container until it is full before moving to the next container.

Test Case 1:

Input:

n = 7

weights = [5, 10, 15, 20, 25, 30, 35]

max_capacity = 50

Output: 4

Test Case 2:

Input:

n = 8

weights = [10, 20, 30, 40, 50, 60, 70, 80]

max_capacity = 100

Output: 6

```
main.py
1 def min_containers(weights: list[int], max_capacity: int) -> int:
2     if not weights:
3         return 0
4
5     containers = 1
6     current_load = 0
7
8     for w in weights:
9         if current_load + w <= max_capacity:
10             current_load += w
11         else:
12             containers += 1
13             current_load = w
14
15     return containers
16
17 # Test Verification
18 print(min_containers([5, 10, 15, 20, 25, 30, 35], 50)) # Output: 4
19
```

input

4

11. Given a graph represented by an edge list, implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) and its total weight.

Test Case 1:

Input:

n = 4

m = 5

edges = [(0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4)]

Output:

Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]

Total weight of MST: 19

Test Case 2:

Input:

n = 5

m = 7

edges = [(0, 1, 2), (0, 3, 6), (1, 2, 3), (1, 3, 8), (1, 4, 5), (2, 4, 7), (3, 4, 9)]

Output:

Edges in MST: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)]

Total weight of MST: 16

```
main.py
7         return i
8     self.parent[i] = self.find(self.parent[i])
9     return self.parent[i]
10
11     def union(self, i, j):
12         root_i = self.find(i)
13         root_j = self.find(j)
14         if root_i != root_j:
15             self.parent[root_i] = root_j
16             return True
17         return False
18
19     def kruskal_mst(n: int, edges: list[tuple]) -> tuple[list[tuple], int]:
20         sorted_edges = sorted(edges, key=lambda item: item[2])
21         dsu = DSU(n)
22         mst_edges = []
23         total_weight = 0
24
25         for u, v, w in sorted_edges:
26             if dsu.union(u, v):
27                 mst_edges.append((u, v, w))
28                 total_weight += w
29
30         return mst_edges, total_weight
31
32     # Test Verification
33     edges1 = [(0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4)]
34     print(kruskal_mst(4, edges1)) # Output: ([ (2, 3, 4), (0, 3, 5), (0, 1, 10) ], 19)
```

input

[(2, 3, 4), (0, 3, 5), (0, 1, 10)], 19)

12. Given a graph with weights and a potential Minimum Spanning Tree (MST), verify if the given MST is unique. If it is not unique, provide another possible MST.

Test Case 1:

Input:

n = 4

m = 5

```
edges = [ (0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4) ]
```

```
given_mst = [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
```

Output: Is the given MST unique? True

Test Case 2:

Input:

```
n = 5
```

```
m = 6
```

```
edges = [ (0, 1, 1), (0, 2, 1), (1, 3, 2), (2, 3, 2), (3, 4, 3), (4, 2, 3) ]
```

```
given_mst = [(0, 1, 1), (0, 2, 1), (1, 3, 2), (3, 4, 3)]
```

Output: Is the given MST unique? False

Another possible MST: [(0, 1, 1), (0, 2, 1), (2, 3, 2), (3, 4, 3)]

Total weight of MST: 7

```
main.py
44     if dsu.union(u, v):
45         alternative_edges.append((u, v, w))
46
47         # If we successfully created a valid MST with the exact same total weight
48         if len(alternative_edges) == n - 1 and compute_weight(alternative_edges) == given_weight:
49             return False, alternative_edges
50
51     return True, given_mst
52
53
54 # --- Test Case 1 ---
55 edges1 = [(0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4)]
56 given_mst1 = [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
57
58 is_unique1, mst_res1 = verify_mst_uniqueness(4, edges1, given_mst1)
59 print(f"Test Case 1 - Is unique? {is_unique1}")
60 print(f"MST Edges: {mst_res1}\n")
61 # Expected: True
62
63 # --- Test Case 2 ---
64 edges2 = [(0, 1, 1), (0, 2, 1), (1, 3, 2), (2, 3, 2), (3, 4, 3), (4, 2, 3)]
65 given_mst2 = [(0, 1, 1), (0, 2, 1), (1, 3, 2), (3, 4, 3)]
66
67 is_unique2, mst_res2 = verify_mst_uniqueness(5, edges2, given_mst2)
68 print(f"Test Case 2 - Is unique? {is_unique2}")
69 print(f"Another possible MST: {mst_res2}")
70 # Expected: False, followed by an alternate edge combination
71
```

input

```
Test Case 1 - Is unique? True
MST Edges: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]

Test Case 2 - Is unique? False
Another possible MST: [(0, 1, 1), (0, 2, 1), (2, 3, 2), (3, 4, 3)]
```